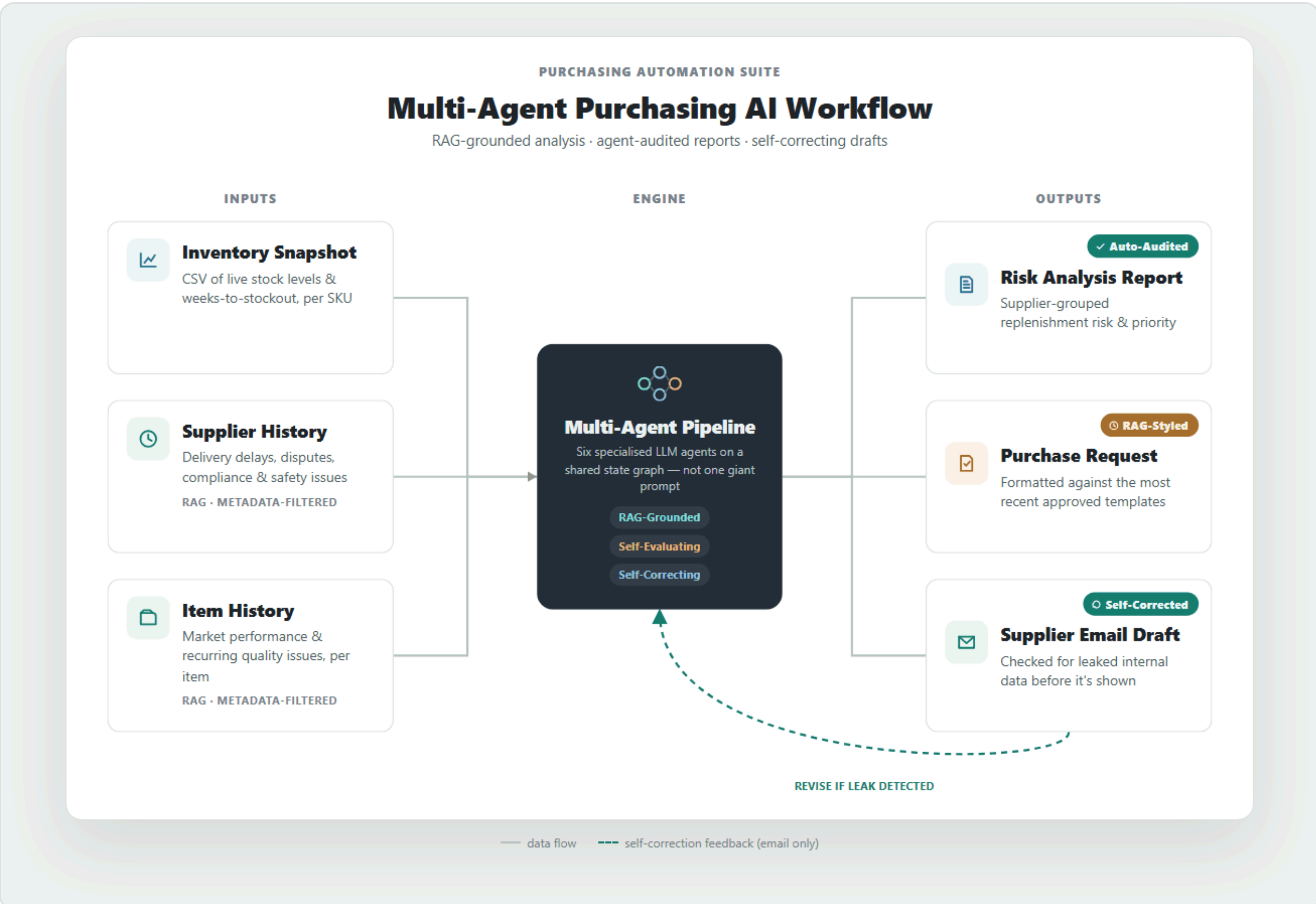
Multi-agent Purchasing AI Suite

A multi-agent LLM pipeline that jointly analyzes an inventory snapshot together with supplier and item history context via RAG, automating the manual work previously handled by purchasing staff — **from inventory-risk analysis and replenishment planning to drafting analysis reports, purchase request forms, and supplier emails**.

Rather than stopping at a simple stock check, it uses RAG to automatically retrieve and analyze relevant context scattered across many documents, including supplier delivery performance and delay history, transaction and negotiation records, customs/regulatory/safety issues, item market performance, and item-specific issue history. Each output is then **independently audited by a separate agent for data and logical consistency, producing a quality-evaluation report alongside it**. The email draft also goes through a **self-correction loop that checks for internal information leakage and rewrites the draft if any risk is found**.



Demo: https://soominmyung.com/purchasing-automation

Source: https://github.com/soominmyung/purchasing-automation

At a Glance

Period	Oct 2025 – Dec 2025
Role	Solo project — architecture, development, deployment
Domain	Generative AI-based inventory-risk analysis & automated document generation
Core stack	Python, FastAPI, LangGraph, ChromaDB, GPT-4o, Docker, GCP Cloud Run
Starting point	n8n low-code prototype → re-architected as a Python service

The Problem

Purchasing staff review hundreds of SKU-level inventory records and, on that basis, spend considerable manual effort writing documents such as analysis reports, purchase request forms, and supplier emails.

The challenge is that this process is **highly dependent on staff availability and experience**. **When time pressure or limited familiarity with a supplier or item prevents a full review** of evidence scattered across many documents — a supplier's delivery and quality history, past negotiation terms, customs/regulatory/safety issues, and an item's market performance — **the result can be inefficient ordering and potential margin loss**.

What It Does

Given an inventory snapshot, the system automatically produces four outputs.

1. **Purchasing analysis report** — supplier-grouped inventory-risk assessment and replenishment planning
2. **Quality-evaluation report** — a separate agent audits the analysis for data and logical consistency
3. **Purchase request form (PR)** — formatted per supplier for the approval process
4. **Supplier email draft** — outbound communication for delivery and stock-availability inquiries

The inventory snapshot is provided as a CSV of items flagged for potential shortage, while supplier and item history documents are uploaded, embedded into a vector DB, and then automatically retrieved during the analysis stage.

Architecture

PoC-to-Production — from n8n prototype to production code

I first validated the concept quickly with an **n8n** low-code workflow. By wiring nodes together, this stage confirmed the feasibility of the "CSV → grouping → LLM analysis → document generation" flow.

Once validated, I re-architected the entire workflow in **Python (FastAPI + LangGraph)** to implement complex branching logic, state management, and container deployment. In the process I moved the node-based flow onto an async API and a state graph, refined it into a deployable service with API-key authentication and streaming, and set up CI/CD (GitHub Actions → Cloud Run) for the public demo (actual operations run on the company's local server).

Multi-agent pipeline (LangGraph)

Six specialized agents are connected as a LangGraph state graph, and the validator node loops back to an earlier step to rewrite when needed.

INPUT Inventory snapshot (CSV) + supplier/item context

01 **Supplier grouping & order date/quantity calculation**
Back-calculated from an 18-week lead time · targets 26 weeks of stock coverage after delivery
Deterministic Python formula, no LLM

02 **Analysis agent**
Inventory-risk analysis
RAG: metadata filter, then latest N history items

03 **Analysis Evaluation agent**
Analysis result quality audit (logical & data consistency)
Scores accuracy · faithfulness (hallucination) · reasoning · suitability

04 **Analysis Report agent**
Purchasing analysis report authoring
RAG: latest N approved templates

05 **PR Content agent**
Purchase request content generation
RAG: latest N approved templates

06 **PR Document agent**
Formal purchase request document authoring
RAG: latest N approved templates

07 **Email agent**
Supplier email draft
RAG: latest N approved templates

08

Validator node

Sensitive-info (inventory figures, analysis details, etc.) leak check → auto-rewrite loop

Guardrail against sensitive-info leaks, precise rewrite instructed via failure-reason feedback

Why multi-agent

Compared to handling everything with a single monolithic prompt, this design offers:

- **Modularity** — each step can be swapped or improved independently without touching the others; in practice, I isolated just the analysis agent to test replacing GPT-4o with a self-hosted Llama-3-8B model.
- **Early error containment** — because the analysis evaluation agent independently audits the analysis, a flawed analysis is caught before it propagates into the downstream steps (PR, email).
- **Precise self-correction and validation** — when the validator node detects a problem, it can pinpoint exactly which step needs a rewrite instead of re-running the entire pipeline from scratch.

Core Components

RAG knowledge base (ChromaDB)

- **Chunking & Storage** — supplier-record and item-history PDFs (uploaded individually or in bulk as ZIP) are chunked (1,000 chars, 200-char overlap), converted to OpenAI embeddings, and stored in five independent collections split by document type (supplier history, item history, and report/PR/email examples).
- **Chunk Size & Overlap Tuning** — reviewing the actual supplier/item history documents, I found they typically run about 1–2 A4 pages, and used that as the basis to benchmark chunk sizes of 500, 1,000, and 1,500 characters for storage/retrieval efficiency. A too-small chunk size (500 chars) caused context loss, while a too-large one (1,500 chars) caused context blending across different events, so I chose 1,000 characters as the optimal size, and set a 20% (200-char) overlap so that context spanning a chunk boundary isn't lost.
- **Workflow-tailored design** — building on the observation that the actual supplier/item history documents follow a convention of stating the supplier name and item code at the top, I designed **reliable metadata filtering using regex alone**, without any complex ontology implementation.
    - **Metadata tagging** — at ingest time, regex extracts the supplier name and item code and attaches them as document metadata. Since every child chunk inherits the parent document's metadata after chunking, the filter always applies correctly even when a given chunk's text doesn't literally contain that name.
    - **Explicit metadata filtering + most-recent-first retrieval (two-stage retrieval)** — when the analysis agent looks up history, it passes the actual supplier name and item code the pipeline already knows as a metadata filter, narrowing the candidate set to documents belonging to that supplier/item only. Since filtering alone already leaves nothing but documents that genuinely belong to that supplier/item, it then fetches the top N by **the document's own recorded date (event_date)** — no similarity ranking involved. If date extraction fails, it falls back to the ingest timestamp. Other filtering strategies can, of course, be swapped in depending on the retrieval purpose.

    - **Strength** — pure semantic similarity search risks pulling in similar narratives from other suppliers (e.g. comparable shipping-delay cases), but the metadata filter blocks this cross-contamination at the source. Building on an existing documentation convention, this was achieved without implementing any ontology or knowledge graph.

- **Purpose-differentiated retrieval strategy** — the supplier/item history collections and the example (analysis report/PR/email best-practice) collections both fetch the top N by the document's own date (event_date). What differs is **whether they filter before ranking**, and that follows from what each is for.
    - **History — filter first, then rank** — this retrieval is after **facts about one specific supplier/item**, so it must narrow by metadata before ranking by date.
    - **Examples — rank across the whole collection** — these are a **style reference** for how the company's format has recently changed, so there is no need to narrow to a specific entity.

  Both collections therefore parse a date from the document text at ingest time and store it as event_date (falling back to the ingest timestamp if parsing fails); **only the history collections layer a metadata filter on top**.

- **Separating content-structuring agents from documentation agents** — both flows (analysis → report, PR content → PR document) follow one design principle: **the agent that structures the content is separate from the agent that formats it. Loading a single prompt with both content reasoning and format compliance would make it heavy to control and maintain, harder to manage hallucination, and — from a RAG standpoint — would tangle content-judgment references with format references.**
  The split shows up in what each side retrieves: the structuring agents (analysis / PR content) reference **what belongs in each field** (e.g. how specific the justification or buyer-check items should be), while the documentation agents (report / PR document) reference **only how the document is formatted** (header structure, tone). To stop the structuring agent from lifting an example's content instead of its judgment criteria, the prompt explicitly instructs it to reference structure only and never copy content — verified by testing across different risk types (shipping delays, price spikes, quality issues) to confirm example content does not leak through.

AI analysis quality-evaluation (Evaluator) agent

A separate analysis evaluation agent cross-references the analysis agent's output (JSON) against the source supplier/item history it was based on, scoring it out of 10 on four criteria — **data accuracy, history faithfulness (hallucination check), logical reasoning, and operational suitability**. Because it compares against the exact history retrieved by the analysis agent — passed directly rather than re-fetched — it avoids the failure mode of misjudging a genuinely grounded statement as "unsourced" and wrongly scoring it as a hallucination.

Guardrail — self-correction loop

The email output passes through the validator node, which checks for sensitive-info leakage (inventory figures, analysis details, etc.). If a leak is detected, it is not returned as-is; the graph loops back to regenerate the draft.

Real-time streaming (SSE)

Instead of a single request-response, the progress of each pipeline stage (CSV parsing → grouping → analysis → document generation) is streamed to the client in real time via Server-Sent Events, showing the current pipeline stage live and surfacing errors immediately when they occur.

Hybrid design — separating deterministic computation from LLM reasoning

In the supplier-grouping stage, the recommended order date and quantity are **computed with a Python formula** based on `WksToOOS` (weeks to out-of-stock) and `CurrentStock`. Accuracy-critical values like dates and quantities are handled by deterministic code rather than left to LLM reasoning, eliminating hallucination risk; the LLM then takes that result and focuses solely on risk interpretation and document authoring over the prioritized, structured data.

Security & Observability

Security: the public demo has header-based API-key authentication (`X-API-Key`) and per-IP rate limiting to prevent token abuse by anonymous users. The in-house deployment implements per-user intranet ID/PW login. Observability: LangSmith tracks token usage, latency, and prompt performance for every node, and custom logic outside LangChain is also instrumented with `@traceable`.

Deployment

The public demo runs as a Docker container on GCP Cloud Run (serverless, scale-to-zero) so anyone can access it, with CI/CD via GitHub Actions building and auto-deploying the image on every push to `main`. For actual in-house operations, the app runs continuously on the same 24/7 local server as SAP ERP, so the team uses it over the intranet — an environment with no cold-start or vector-store volatility issues.

Fine-tuning Study: SFT + DPO on Llama-3-8B

I tested whether the GPT-4o analysis agent could be replaced with a self-hosted model. **The goal was to reduce per-call cost and data-exposure risk**, and the approach was to distill GPT-4o's knowledge into Llama-3-8B.

Training data

- **Data source** — instead of real company data, I used **synthetic purchasing scenarios** generated by GPT-4o.
- **Scenario composition** — each consists of inventory rows (item code, item name, supplier, risk grade, current stock, weeks-to-out-of-stock) and supplier/item history (natural language).
- **Diversity** — a wide variety of history patterns such as shipping delays, price spikes, quality issues, and demand fluctuations.
- **Adversarial cases included** — I deliberately included cases like data contradictions (high stock yet imminent stockout), conflicting context (a “trusted” label vs. a recent failure), ambiguous history, and missing values, to improve robustness. A model trained on such cases, when faced with similar production cases, responds by **explicitly stating contradictions or gaps as such** rather than masking them or fabricating information.

- **Knowledge distillation** — the analysis GPT-4o produced for these scenarios (JSON: analysis report, critical questions, supplementary timeline) was treated as ground truth for Llama to reproduce.
- **Fine-tuning scope** — limited to the **output of the analysis agent** being replaced, not the whole pipeline.

Stage 1 — Supervised Fine-Tuning (SFT)

- Llama-3-8B + QLoRA (4-bit, LoRA rank 16 — ~41M trainable params / 8B, 0.51%)
- 30 teacher examples (analysis JSON GPT-4o generated per scenario)
- **Training-data shape**: JSONL, `{instruction, input: {inventory, supplier_history, item_history}, output: {analysis: {...}}}` — only `output.analysis` (analysis report, critical questions, supplementary timeline) is used as the label
- Vertex AI, 1× Tesla T4, 5 epochs, 367s
- Training loss: 1.14 → 0.41

Stage 2 — Direct Preference Optimization (DPO)

- 25 preference pairs (holdout of 5 excluded)
- **Preference-pair construction** — `chosen` is fixed to the GPT-4o ground-truth analysis JSON (distillation requires an external reference point beyond the model's own ability), and `rejected` is selected by having the SFT model generate 4 candidates per prompt at temperature 0.8, scoring them with a GPT-4o judge, and **taking only the lowest-scoring one**. **The judge was explicitly instructed to evaluate only data accuracy and reasoning quality — not style or length — so that stylistic differences would not leak into the preference signal**.
- **Training-data (preference-pair) shape**: JSONL, `{prompt, chosen, rejected}` — `prompt` uses the same template as SFT (Response excluded), and **the rationale (judge score and comment) is stored alongside each `rejected` to enable post-hoc analysis**.
- Vertex AI, Tesla T4, 3 epochs, ~14 min
- Unsloth `PatchDPOTrainer()` + TRL `DPOTrainer`

Evaluation (GPT-4o-as-judge, 5-example holdout)

<table>
<tr><th>Model</th><th>Avg. score</th><th>Valid JSON</th><th>Scoring criteria</th></tr>
<tr><td>Base Llama-3-8B</td><td>0.0 / 10</td><td>0%</td><td rowspan="4">GPT-4o scores two criteria out of 10 each and averages them: (1) data accuracy — did it accurately reflect input values such as supplier name, item code, stock, and risk grade; (2) reasoning quality — is the replenishment analysis and are the critical questions logically sound. (This is a simplified benchmark-only scheme, separate from the production analysis quality-evaluation agent's four criteria.)</td></tr>
<tr><td>Llama-3-8B SFT</td><td>9.5 / 10</td><td>100%</td></tr>
<tr><td>Llama-3-8B SFT + DPO</td><td>8.5 / 10</td><td>100%</td></tr>
<tr><td>GPT-4o (reference)</td><td>10.0 / 10</td><td>100%</td></tr>
</table>

Interpreting the results

- **SFT result** — reached about 95% of GPT-4o's evaluation quality and produced valid structured output on every example. This supports the feasibility of replacing the GPT-4o analysis agent with a self-hosted model.
- **DPO regression** — despite training on preference pairs whose quality was actually verified by a judge, it *dropped* relative to SFT (9.5 → 8.5).
    - **Root-cause analysis** — the judge comments show that most of the regressed examples were flagged for "the critical-questions field being missing." Looking into the training data, `chosen` (GPT-4o) includes this field in all 25 cases, while `rejected` includes it in only 7 of 25 (28%) — so most of the training signal should have pushed toward *including* the field. This opposite result suggests that **DPO's contrastive learning only adjusts the relative probability of the whole sequence and cannot pinpoint "which part is responsible" (a lack of credit assignment)**. The seven exception cases likely acted as noise, driving generalization in the wrong direction at this small sample size (25).
    - **Conclusion** — even with judge-verified preference pairs, DPO could not guarantee a stable improvement over SFT at this scale (25 pairs). **Given that SFT already reached 95% of GPT-4o's quality, adopting SFT alone is the more reasonable conclusion at this project's scale than investing further to close the remaining gap with DPO.**

Inference Serving & Quantization (vLLM)

The fine-tuning study produced a self-hostable analysis model; this stage measures **how efficiently that model can actually be served** — the deciding factor in whether the switch from GPT-4o to self-hosting is worth making. I served the fine-tuned Llama-3-8B (base + LoRA) on a single **NVIDIA L4 (24 GB, GCP `g2-standard-8`)** via the official **vLLM** OpenAI-compatible container, and compared it against the plain HuggingFace `generate()` loop used in the earlier evaluation.

Continuous batching — throughput rises without added latency

Concurrency	Throughput (tok/s)	p50 latency (s)
1	16.1	7.94
8	122.8	8.31
16	**242.2**	8.44
32	240.5	8.45

Going from concurrency 1 to 16, **throughput rises about 15× (16 → 242 tok/s) while per-request latency stays at roughly 8 seconds** — the benefit of continuous batching, which processes many requests on the GPU in parallel. Past about 16 the GPU's compute is saturated, so adding concurrency no longer increases throughput.

Comparison with the previous inference path, and FP8 quantization

- **Throughput vs. the previous path** — on the same GPU, the sequential HuggingFace `generate()` calls used in the earlier evaluation managed 11 tok/s, while serving with vLLM raised throughput to 242 tok/s, **about 22×**.
- **FP8 quantization** — loading the model in 8-bit (a single vLLM flag, no separate converted checkpoint, using the L4's FP8 compute units) pushed throughput to **393 tok/s**. That is **about 1.6×** over FP16, and because the weights take up less memory, KV-cache capacity grew from **21k to 81k tokens (about 3.9×)** — headroom for longer contexts or more concurrent requests.
- **Checking for quality loss from quantization** — to verify that 8-bit serving does not degrade output, I scored the FP16 and FP8 outputs for all 30 scenarios with this project's GPT-4o evaluation method. The result was **7.2 → 7.1 out of 10 (−0.1)**, i.e. no difference.

Takeaway

On a single GPU (L4), throughput went from **11 → 242 → 393 tok/s, roughly 36× overall**. FP8 quantization delivered both higher throughput and more memory headroom without costing quality. Together with the fine-tuning results, this confirms the feasibility of replacing GPT-4o with a local model on both quality and performance grounds.

Outcomes

- **Higher automation and analysis quality** — automated repetitive SKU-level inventory review and document authoring, standardizing analysis quality that previously varied by person and situation.
- **RAG-based context analysis** — multi-source context such as supplier history and lead times, which can be missed in manual review under time and experience constraints, is pulled in and combined within seconds by RAG, supporting risk-priority-based decisions quickly and consistently.
- **Validated self-hosting feasibility** — demonstrated that the fine-tuned local model (SFT) reaches about 95% of GPT-4o's quality. Confirmed two concrete benefits of a self-hosting switch: eliminating per-call API costs and keeping data internal for privacy.
- **Operations-oriented design** — SSE real-time streaming (live progress even while waiting, avoiding timeout risk), a self-correcting agent graph (automatic sensitive-info leak detection and rewrite), and LangSmith-based observability (per-node token/latency/prompt-performance and cost monitoring plus root-cause tracing).
- **Accuracy and reliability** — separated roles so that accuracy-critical computation (order date, quantity) is handled by deterministic formulas while context-dependent judgment is handled by LLM reasoning.
- **Workflow-tailored efficiency design** — after recognizing the actual document-authoring convention (supplier name and item code stated at the top), I solved RAG-retrieval cross-contamination (documents from other suppliers or items being mixed in) with regex metadata extraction + forced filtering alone, without heavy structuring like an ontology. Rather than applying a generic engineering pattern as-is, I observed how the work is actually done and designed a lightweight, efficient solution around it.

Future Work

GPT-4o → self-hosted model migration

The analysis agent currently relies on GPT-4o calls, incurring per-call cost and sending data externally. To reduce this, I validated a self-hosted model and confirmed migration feasibility with SFT alone reaching about 95% of GPT-4o's quality (on Llama-3-8B). DPO was additionally tested with judge-verified preference pairs but yielded no improvement over SFT at this scale, so I am proceeding with **SFT alone** for now. That said, the following remain before an actual migration:

- **Conditions for revisiting DPO** — the regression here appears to stem from DPO's contrastive learning being unable to pinpoint “which part is responsible” (a credit-assignment limitation) compounded by the small sample size of 25 pairs. It could be revisited by **scaling up**, by reusing the existing four-criterion analysis evaluation-agent structure to break the preference signal down per field/criterion as a **multi-attribute reward**, or by switching to **Best-of-N filtering + SFT**, which eliminates contrastive (push-down) learning altogether and thereby removes the side-effect risk at the source. For now, though, at this scale (25 pairs, 5-example holdout) the remaining gap (5%) is too small to justify that engineering investment, making SFT-alone the more reasonable choice.
- **Evaluating newer models** — in this project GPT-4o plays two roles: the production analysis agent (repeated calls, cost-sensitive) and the teacher that produces the training data (one-off calls, cost-insensitive). The teacher side could be swapped for a higher-tier model like GPT-5 — a one-off cost with little burden — to raise training-data quality, while the student (replacement) side could benchmark, beyond Llama-3-8B, newer open-source small models released since (Qwen, newer Llama versions) to improve production cost and performance together.

RAG retrieval extension idea — evaluating ontology / knowledge graph

The current approach (metadata filter + most-recent-first retrieval) is sufficient for its core purpose: finding relevant documents quickly and precisely at the supplier/item level. To go one step further and **support relational/aggregate queries that span multiple documents** — e.g. “Among the other items this supplier supplies, which have had similar quality issues in the past?” — an ontology / knowledge graph could be added.

- **Technical approach — ontology design → knowledge graph implementation** — first, entity types like “supplier” and “item” and relation types like “supplies” and “had a quality issue” are **defined as an ontology**. Then a **lightweight knowledge graph (e.g. Neo4j)** populated with real data following that schema is used alongside vector search. **The existing metadata-filter + most-recent-first retrieval keeps handling “finding documents about this supplier/item,” while “traversing to other items/issues connected to this supplier along the relations the ontology defines” is handled by the newly added knowledge graph** — dividing the roles this way.
- **Expected benefit — multi-hop reasoning & aggregation** — the system is currently strong at questions answerable within a single document (what is this supplier's history), but it would then also answer accurately for questions requiring several documents/entities to be chained (“Among the other items this supplier supplies, have any had problems?”) and for count-based questions (“How many suppliers had two or more delays this quarter?”).

- **Adoption condition — incremental rollout** — ontology design (defining entity/relation types) and extracting relations from data to match that schema require additional work. So the realistic path is to confirm that such “cross-document questions” actually come up often in real use, and then add only as much as needed, incrementally.

Tech Stack

Area	Technology
Backend	Python, FastAPI (async, SSE)
Orchestration	LangGraph (state-based multi-agent), LangChain
LLM	GPT-4o / GPT-4o-mini; Llama-3-8B (QLoRA)
Fine-tuning	Unsloth, TRL (SFTTrainer / DPOTrainer), Vertex AI, W&B
Inference serving	vLLM (containerized, continuous batching), FP8 quantization, NVIDIA L4
Vector DB	ChromaDB (RAG)
Observability	LangSmith
Frontend	React, TypeScript, Framer custom code components
Data processing	Pandas, PyPDF, python-docx
Infra / CI-CD	Docker, GCP Cloud Run, Vertex AI, Artifact Registry, GitHub Actions

---

Soomin Myung · msm1640@gmail.com · +82 10-5600-4620